

Auditoría profunda de diseño, calidad y riesgos

Sistema EIEInfo - Entrega 2

IE-0417 - Diseño de Software

Junio 2026

Pregunta guía

¿Cuáles son los principales problemas de diseño del sistema, por qué importan y cómo se relacionan entre sí?

- Se auditó el sistema EIEInfo desde su código y configuración.
- La revisión se enfocó en diseño, calidad, riesgos y evidencia concreta.
- El resultado principal es una auditoría profunda con 14 hallazgos priorizados.

El problema principal

El riesgo no es que EIEInfo sea un monolito. Para una organización pequeña o mediana, un monolito puede ser razonable.

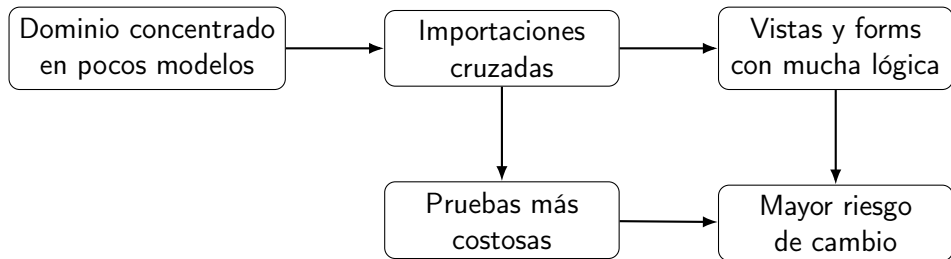
El riesgo real

El riesgo aparece porque el monolito tiene acoplamiento interno alto: muchos módulos dependen directamente de modelos, vistas, formularios y reglas internas de otros módulos.

- EIEInfo es una aplicación web Django.
- Esta dividida en apps internas: estudiantes, profesores, administrativos, cursos, firma_digital, trabajos_finales, entre otras.
- Se ejecuta como una aplicación única, pero puede desplegarse con varios contenedores.
- La base de datos esperada es MySQL/MariaDB.
- Nginx y Gunicorn forman parte del despliegue.

- Archivo base: `src/server/eieinfo/urls.py`.
- El sistema enruta portales completos:
 - `/estudiantes/`
 - `/profesores/`
 - `/administrativos/`
 - `/cursos/`
 - `/firma_digital/`
 - `/trabajo_final_de_graduacion/`
- Esto confirma una arquitectura monolítica modular.

Mapa conceptual del riesgo



- Mantenibilidad
- Modificabilidad
- Cohesión
- Acoplamiento
- Testabilidad
- Seguridad básica
- Consistencia arquitectónica
- Observabilidad operativa

E2-H01 - Severidad alta

administrativos concentra conceptos institucionales que no son solo administrativos.

- Incluye Funcionario, Ciclo, Lugar, Horario, Nombramiento, Jefatura, Comision y roles.
- Otros módulos dependen de esas entidades.
- Consecuencia: cambiar un concepto central puede impactar varias áreas del sistema.

E2-H02 - Severidad alta

`profesores/views/consejo_asesor.py` concentra demasiada coordinación.

- Archivo con 1810 líneas.
- Importa modelos y formularios de varios dominios.
- Mezcla consultas, validaciones, mensajes, reportes y redirecciones.
- Riesgo: cambios de horarios, aulas, nombramientos o permisos son más difíciles.

Hallazgo 3: autenticación no uniforme

E2-H03 - Severidad alta

Los usuarios no se autentican de forma completamente uniforme.

- Profesores y administrativos usan `django_login`.
- Estudiantes manejan variables de sesión propias.
- También existen campos propios de contraseña en modelos del dominio.
- Consecuencia: permisos, auditoría y mantenimiento de sesiones se vuelven más complejos.

E2-H04 - Severidad alta

Existen credenciales y claves sensibles dentro del repositorio.

- Evidencia: `docker/django/secret_credentials.py`.
- Contiene `SECRET_KEY`, credenciales de base de datos, SMTP y claves de servicios.
- Aunque algunos valores parecen placeholders, el patrón sigue siendo riesgoso.
- Recomendación: variables de entorno o gestor de secretos.

Hallazgo 5: configuración por hostname

E2-H05 - Severidad alta

La configuración de producción depende del nombre del servidor faraday.

- En `settings.py`, producción se activa por `socket.gethostname()`.
- Si no es faraday, se usa `ALLOWED_HOSTS=['*']`.
- No es un problema porque el servidor actual exista; es un riesgo al migrar o cambiar infraestructura.
- Recomendación: variables como `DJANGO_DEBUG` y `DJANGO_ALLOWED_HOSTS`.

- **E2-H06:** dominio TFG dividido entre dos módulos cercanos.
- **E2-H07:** modelos de dominio extensos.
- **E2-H08:** reglas de negocio en formularios y vistas.
- **E2-H09:** dependencias directas entre apps internas.
- **E2-H10:** documentación técnica desactualizada.
- **E2-H11:** dependencias históricas o personalizadas.
- **E2-H12:** pruebas existentes pero costosas de aislar.

Archivo	Líneas
firma_digital/tests.py	2348
profesores/views/consejo_asesor.py	1810
profesores/tests.py	1607
estudiantes/models.py	1348
trabajos_finales/models.py	1313
administrativos/models.py	1179

- administrativos: núcleo institucional compartido.
- profesores: profesor y Consejo Asesor.
- estudiantes: identidad estudiantil y procesos académicos.
- cursos: cursos, grupos, horarios y planes.
- trabajos_finales/trabajo_final_de_graduacion: procesos de graduación.
- firma_digital: documentos, revisión y firma.

- Personas: Funcionario, Profesor, Administrativo, Estudiante, User.
- Estructura académica: Ciclo, Curso, Grupo, Horario, Lugar, PlanDeEstudio.
- Gestión institucional: Nombramiento, Jefatura, Comision, Roles.
- Procesos: asistencias, práctica profesional, proyecto eléctrico, TFG y firma digital.

- Hay pruebas importantes en `firma_digital`, profesores, administrativos, webpage y TFG.
- También existe automatización con `.drone.yml`.
- El riesgo no es ausencia total de pruebas.
- El riesgo es que el acoplamiento obliga a preparar mucho contexto para probar reglas pequeñas.

Recomendaciones priorizadas

- ➊ Extraer un dominio compartido para conceptos institucionales.
- ➋ Refactorizar progresivamente `consejo_asesor.py`.
- ➌ Unificar autenticación y manejo de sesiones.
- ➍ Sacar secretos del repositorio.
- ➎ Actualizar documentación técnica.
- ➏ Definir frontera formal entre proyecto eléctrico y TFG.
- ➐ Agregar pruebas unitarias para reglas extraídas.

Conclusion

EIInfo no necesita dejar de ser monolito para mejorar. La evolución más razonable es convertirlo en un monolito modular con fronteras internas más claras.

- Menos acoplamiento interno.
- Reglas de negocio más localizables.
- Pruebas más pequeñas y mantenibles.
- Configuración y secretos más seguros.